

# Orientação a objetos

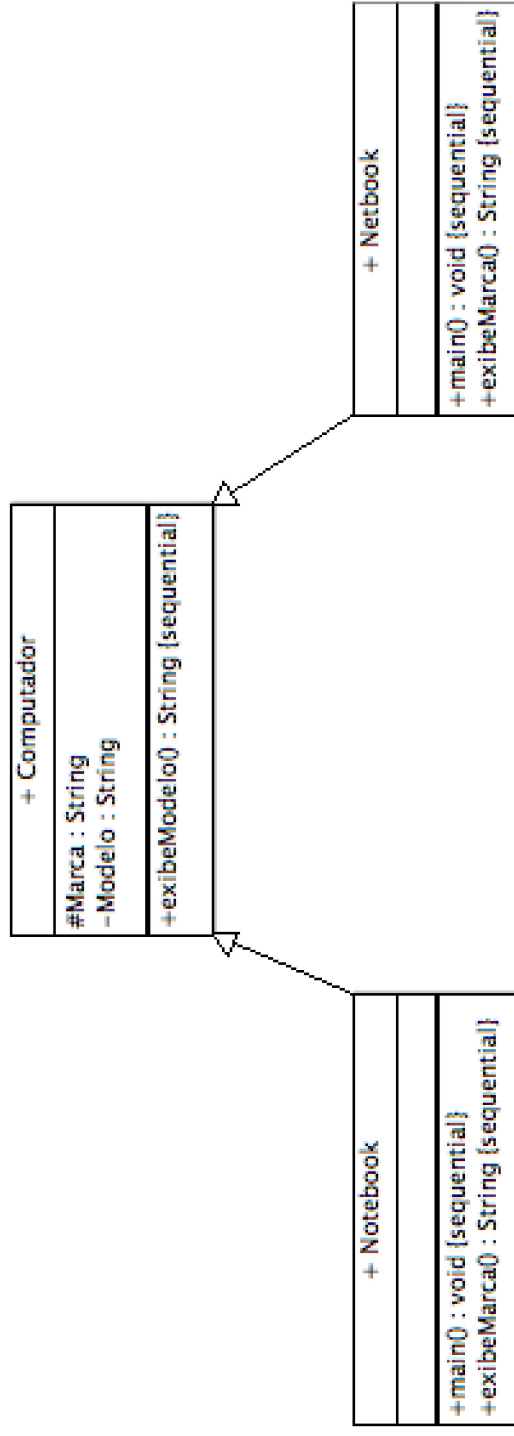
---

Herança e Polimorfismo

# Exercícios

---

- 1) Escreva um programa orientado a objetos baseado no diagrama de classes da UML apresentado abaixo:



- Dica: no método main das classes herdeiras, crie a opção do usuário in marca do computador. Já, o modelo será sempre “Portátil”.

# Exercícios

---

- 2 - Crie uma classe Produto com um método desconto(). Em seguida, crie uma classe ProdutoComDesconto que herda da classe Produto e sobrescreve o método desconto() para calcular o desconto do produto com base em um valor predefinido e imprimir o preço com desconto.
- 3 - Crie uma classe Funcionario com um método calcularSalario(). Em seguida, crie uma classe Gerente que herda da classe Funcionario e sobrescreve o método calcularSalario() para calcular o salário do gerente com base em um bônus e imprimir o resultado.
- 4 - Crie uma classe ContaBancaria com um método depositar(double valor) que adiciona o valor passado como parâmetro ao saldo da conta. Sobrecarregue o método depositar() para aceitar um objeto Cheque e adicionar o valor do cheque ao saldo da conta.
- 5 - Crie uma classe Produto com um método calcularPrecoFinal(double preco) que retorna o preço final do produto com base no preço passado como parâmetro. Sobrecarregue o método calcularPrecoFinal() para aceitar um objeto Cliente e calcular o preço final do produto com base no preço do produto e o desconto do cliente.

# Exercícios

---

- 6) Crie uma classe base `Funcionario` com atributos como nome e salário. Derive classes específicas como `Gerente` e `Desenvolvedor`. `Gerente` possui um bônus enquanto `Desenvolvedor` tem horas extras.
- Implemente métodos sobrecarregados `aumentarSalario` que aumentam o salário baseado em diferentes critérios (porcentagem fixa para todos e uma porcentagem adicional para `Gerente` que considera o bônus).
- Sobrescreva o método `toString` para refletir informações específicas de cada tipo de funcionário.
- Crie um array de `Funcionario` que inclua `Gerente` e `Desenvolvedor`, e demonstre a aplicação dos aumentos de salário e a impressão das informações.

# Exercícios

---

- 7) Crie uma classe `Notificacao` com um método `enviar`. Derive classes `NotificacaoEmail` e `NotificacaoApp` que estendem `Notificacao`.
- Sobrecarregue o método `enviar` em `NotificacaoEmail` para aceitar um mais destinatários.
- Sobrescreva o método `enviar` em cada classe derivada para implementar lógica específica de envio.
- Demonstre o envio de diferentes tipos de notificações usando objetos classes derivadas.

## Exercícios

---

- 8) Crie uma classe Reserva com métodos para adicionar e cancelar reservas. Derive classes como ReservaDeHotel e ReservaDeVoo da classe Reserva.
- Sobrecarregue o método adicionar em ReservaDeVoo para aceitar diferentes tipos de classes (econômica, executiva).
- Sobrescreva o método cancelar em ambas as classes derivadas para tratar políticas de cancelamento específicas.
- Demonstre como criar e cancelar diferentes tipos de reservas usando polimorfismo.

# Exercícios

---

- 9 - Crie uma classe Produto com atributos nome, preco e quantidade. Crie um método para adicionar produtos a uma lista. Em seguida, percorra a lista e imprima os dados de cada produto.
- 10 - Crie uma classe Aluno com atributos nome, nota1 e nota2. Crie uma lista de alunos e adicione alguns alunos nessa lista. Em seguida, percorra a lista e calcule a média de cada aluno. Se a média for maior ou igual a 6, imprima que o aluno foi aprovado. Caso contrário, imprima que o aluno foi reprovado.
- 11 - Crie uma classe Pessoa com atributos nome, idade e sexo. Crie uma lista de pessoas e adicione algumas pessoas nessa lista. Em seguida, crie um método para receber uma lista de pessoas e retorna a quantidade de mulheres. Por fim, crie um método passando a lista de pessoas e imprima a quantidade de mulheres.

## Exercícios

---

- 12 - Crie uma classe Livro com atributos titulo, autor e ano. Crie uma lista de livros e adicione alguns livros nessa lista. Em seguida, ordene a lista de livros pelo ano de lançamento e imprima os dados de cada livro.
- 13- Crie uma classe Conta com atributos numero, titular e saldo. Crie uma lista de contas e adicione algumas contas nessa lista. Em seguida, crie um método que receba uma lista de contas e retorne a conta com o maior saldo. Por fim, chame esse método passando a lista de contas e imprima os dados da conta com o maior saldo.